



MyHealth Care

AI-assisted health records, scheduling & staff workflow

Feature & Implementation Report

Senior Project — University of Bahrain, College of IT

Ali Mohamed Jaafar Mohamed (202208244)

Mohammed A. Redha Meftah (202209027)

Supervisor: Dr. Amal Ghanim

Repository: github.com/Meftah7/myhealthcare_v2

Report generated 31 August 2026

1. Executive summary

MyHealth Care is a cross-platform Flutter application that unifies a patient's medical history, runs an AI-driven appointment scheduler that predicts no-shows, and gives clinical staff a rules- and AI-prioritised task workflow. It is **offline-first**: all data lives in a local SQLite database, the app is fully functional with no network and no API key, and the AI layer degrades gracefully to a deterministic mock when the cloud model is unavailable.

116 / 118

planned tasks complete

3 / 3

research questions answered

74

automated tests passing

~13,200

lines of Dart (excl. generated)

Aspect	Status
Patient portal (history, timeline, vitals, medications, booking)	Complete
AI health summary (RQ1) — mock + live Google Gemini path	Complete
No-show prediction model + AI scheduler (RQ2)	Complete — ROC-AUC 0.75
Staff dashboard, risk detection, task prioritisation (RQ3)	Complete
Admin console (users, departments, analytics, audit, AI settings)	Complete
Windows & Web builds	Release builds green
Android build	Blocked — emulator hypervisor driver (one admin step)
PDF report import (RQ1 stretch)	Deferred — text-extraction plumbing only
Usability study (SUS)	Protocol ready — needs 5–8 participants

2. Research questions & how each is answered

The project proposal defines three problems, each a research question with a matching objective. The codebase maps 1:1 onto them.

RQ1 — Scattered records → a unified, timeline-based health profile

- **Merged health timeline** — medical records (visit notes, labs, imaging, prescriptions, vaccinations, discharge, referrals) and vitals measurements are merged into one reverse-chronological feed, grouped by month, with type filters and free-text search.
- **AI health summary** — an `AIService` produces a narrative summary plus key events, trends, and red flags from a strict JSON contract. Key events appear as inline markers on the timeline; trends deep-link to the matching vitals chart.
- **Record detail** — lab panels render as a table with each value flagged against its reference range (normal / low / high / critical), never colour alone.
- Every AI surface carries a permanent "informational only, not medical advice" banner.

RQ2 — Manual booking & wasted capacity → an AI-driven scheduler

- **No-show prediction model** — a class-balanced logistic regression trained offline in Python on a synthetic 8,500-appointment dataset, exported to JSON, and run as pure-Dart inference on device. 11 features (lead time, prior no-show rate, first-visit, chronic condition, visit type, day/hour, reminders acknowledged, ...).
- **Python ↔ Dart parity** — a test asserts the on-device prediction matches the Python model within 1×10^{-6} on every frozen case, so the reported accuracy numbers are the numbers the app actually uses.
- **Booking wizard** — department → doctor → day, then every open slot is scored and ranked lowest-risk-first, each with a plain-language reason.
- **Explainability** — per-feature contribution breakdown for any prediction.
- **Risk-adaptive reminders** — the reminder schedule escalates with the predicted risk band (1 reminder for low risk, up to 3 with a confirm-or-release request for high risk); idempotent across reschedules.
- The predicted risk and band are stored on the appointment for later analytics.

RQ3 — Staff buried in paperwork → task prioritisation & early risk identification

- **Deterministic rule engine** (runs with AI switched off, no network): severe blood pressure / low SpO₂ / high glucose, critical lab values in the last four months, medication gaps for known chronic conditions, and overdue follow-ups. Each finding gets a severity and a stable de-duplication key.
- **Task generator** — turns open risk flags into rule-scored staff tasks (urgent 0.9 / warning 0.6 / info 0.3) with a due date, idempotent on re-run.
- **AI prioritisation** — an AI score is layered on top and blended with the rule score at read time (weight configurable in settings); the board is fully useful with AI off.
- **Task board** — every task shows its rule score, AI score, due date and a one-line rationale.

Architectural note (stated for the defense): local-only storage means "a doctor opens a patient's chart" happens within one device's database, not across a network. This is handled by a repository layer that all features talk to — a future networked backend is a one-class swap, not a rewrite — and is framed in the report as offline-first architecture, with multi-device sync listed under Future Work.

3. Architecture & technology

Layer	Choice	Notes
Language / SDK	Flutter 3.44.4 / Dart 3.12.2	Windows, Web, Android, iOS targets
State management	Riverpod 2.6.1	Hand-written providers (no code-gen) — deliberate, to keep the build simple and avoid an analyzer version deadlock
Persistence	Drift 2.34 over SQLite	16 tables; native on desktop/mobile, WASM + OPFS on web
Navigation	go_router 17	Role-gated redirect guard; one shell per role
Domain modelling	freezed 3 entities	Immutable entities; <code>Result<T> = Ok Err</code> return type — repositories never throw across the boundary
AI transport	dio 5	Google Gemini (free tier), retry + timeout, always wrapped in a mock fallback
ML training	Python 3.14 / scikit-learn	Offline; exports weights to JSON for pure-Dart inference
Charts	fl_chart 1.2	BP, weight, glucose, heart-rate series
Security	PBKDF2-HMAC-SHA256, flutter_secure_storage	Password hashes carry their iteration count; API key in the OS secure store, never committed

Structure

Feature-first (`lib/features/<feature>/{presentation,application}`) over a three-layer core: `domain` (entities + repository interfaces — the swap point) → `data` (Drift database, repository implementations, synthetic seeder) → `services` (AI, ML, notifications, rules).

Synthetic data

A deterministic seeder builds 5 departments, 12 doctors with weekly schedule templates, 60 patients aged 8–83 with ~2 years of history (records, labs with real reference ranges, vitals, medications, appointments). A hidden per-patient no-show tendency is embedded so the ML model has a real signal to learn. No real patient data — reproducible, no ethics exposure.

4. Feature catalogue

4.1 Patient

Feature	Detail
Sign in / self-registration	Email + password; salted PBKDF2 hashing; session persists across restarts
Home	Next appointment, active medications, AI summary card with regenerate
Health timeline	Merged records + vitals, grouped by month, type filters, search, AI key-event markers
Record detail	Labs as a flagged table vs reference range; source facility; attachments
Vitals	List + manual entry; charts for BP, weight, glucose, heart rate
Medications	Active / past list with dose and frequency
AI health summary	Narrative + key events + trends + red flags; cached on input hash; safety banner
Book appointment	Department → doctor → day → risk-ranked slot list with reasons
My appointments	Upcoming / past; cancel; reschedule
Profile & settings	Contact details, blood type, allergies, chronic conditions

4.2 Staff

Feature	Detail
Dashboard	Today's schedule, unacknowledged risk flags (with acknowledge), task preview, one-tap "scan panel"
Panel scan	Runs the rule engine over every patient, then regenerates the task list
Patient search & list	Free-text over name / national ID
Patient chart	Header (age, conditions, allergies), open flags, medications, recent vitals, timeline
Add clinical note	Free-text visit note, written to the record timeline, audited
Prescribe medication	Name, dose, frequency; appears under the patient's active meds
Enter lab result	Analyte, value, unit, reference range; auto-classified normal/abnormal/critical
Task board	Priority dot, rule & AI score tags, due date, per-task rationale, status menu
AI prioritise	Re-ranks open tasks; blended with the rule score by a configurable weight
Week schedule grid	Seven-day view, week navigation, tap through to the patient chart
Panel analytics	No-show rate, cancellation rate, completed-per-day, upcoming (90-day window)

4.3 Admin

Feature	Detail
User management	Tabbed directory (patients / staff / admins); activate / deactivate; reset password; create staff with department assignment
Departments	Create / rename / describe clinical departments
System analytics	Headline counts + panel-wide appointment statistics
Audit log	Read-only feed of every recorded action (bookings, notes, prescriptions, acknowledgements)
AI settings	API key entry, model selection, mock-mode toggle, AI/rule blend weight, re-seed demo data

4.4 Cross-cutting

- Role-gated routing — patients, staff and admins cannot reach each other's areas.
- Adaptive shell — bottom nav on compact, navigation rail on wider screens.
- Light / dark theme from a single seeded colour scheme; a 10-token clinical status ramp.
- Shared empty / error / loading / confirm widgets used everywhere.
- Every fallible operation returns `Result` ; failures surface as retryable messages, never crashes.

5. Results & evidence

5.1 No-show model (RQ2)

Class-balanced logistic regression, `no-show-v1`. Test set 2,145 appointments, 25.1 % no-show rate.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression	0.733	0.476	0.623	0.540	0.754
Baseline — majority class	0.749	0.000	0.000	0.000	—
Baseline — simple heuristic	0.698	0.433	0.653	0.521	0.683

The model beats both baselines on ROC-AUC and recall — it catches ~62 % of real no-shows, which is the metric that matters for filling wasted slots. Strongest predictors: prior no-show rate (+1.02), lead time (+0.35), first visit (+0.28).

5.2 Performance (debug test VM — relative, not release figures)

Metric	Value
Seed database (12 staff, 60 patients, full history)	~8.2 s
Timeline query (busiest patient, 47 records)	~87 ms
Database file size after seed	~1.0 MB
Mock AI summary (mean of 20)	~258 ms
App cold start to first frame	~2.9 s

5.3 Test coverage

Kind	Covers
Unit — ML	Python↔Dart parity (1e-6), feature extraction & clamping, risk bands, contribution sums
Unit — rules	Risk detection thresholds, severity, de-duplication, task generation & idempotency
Unit — repositories	Auth round-trip, appointment slots & double-booking, lab classification, task ordering, admin user ops, department upsert, appointment range queries — all on an in-memory database
Unit — AI	JSON parsing (clean / fenced / malformed / empty / no-summary), transport failure & timeout → <code>Err</code> not throw, fallback to mock
Unit — security	PBKDF2 verify / reject / random salt / work-factor migration
Widget	Login (3 roles), timeline + vitals, booking, staff dashboard + panel scan
Integration	Seed → patient books an AI-ranked slot → assigned staff logs in and sees it on the week grid
Accessibility	Contrast, Android/iOS tap-target size, labelled tappables, 2× text scale without overflow

74 tests, all passing. `flutter analyze` is clean under strict lints.

5.4 Platform status

Platform	Build	Notes
Windows	Release .exe green	Primary demo target; 74 tests run here
Web (Chrome)	Release build green	Drift on WASM/OPFS; in-app notification fallback
Android	Blocked	Emulator needs the AEHD hypervisor driver (one elevated install) or a physical device; code is platform-clean
iOS	—	Needs macOS + Xcode; declared as a known limitation

6. What was built — phase by phase

Phase	Outcome	Done
0 — Scaffold	Project created for 4 platforms; theme & design system; router with role shells; DI; Result / Failure types; Drift verified on Windows & Web; strict lints; README	13 / 14
1 — Data foundation	16 Drift tables; domain entities; repository interfaces + implementations; DI registration; clinical vocab; deterministic seeder (2y history); ERD exported	22 / 22
2 — Auth + patient core	Password hashing; login; registration; persistent session; role-gated routing; home; timeline + filters + search; record detail; vitals list/entry/charts; medications; profile; shared widgets	15 / 18
3 — AI layer (RQ1)	AiService interface; context builder with token budgeting; mock service; strict JSON contract; live Gemini path with retry + fallback; secure key storage; provider selection; hash-keyed cache; summary screen; timeline markers; trend links; safety banner; provenance	14 / 14
4 — ML + scheduling (RQ2)	Python pipeline (dataset → features → balanced LR → evaluate → export); baseline comparison; Dart feature extractor + predictor + contributions; 1e-6 parity test; slot generator; booking wizard; risk×convenience ranking; cancel/reschedule; stored risk; notification abstraction + in-app fallback; risk-adaptive reminders	18 / 19
5 — Staff + admin (RQ3)	Rule engine (vitals/labs/med-gaps/follow-ups) + severity + de-dup; task generator; staff shell; dashboard; patient search + chart; clinical notes; prescriptions; lab entry; AI task prioritisation; task board with rationale; week grid; panel analytics; user management; departments; system analytics + audit log; AI settings	16 / 17
6 — Testing & results	Unit / widget / integration / accessibility test suites; cross-platform smoke; performance harness; usability-study protocol + SUS instrument	8 / 9

7. Known limitations & remaining work

Item	State	Path to close
Android build verification	Blocked	Run <code>silent_install.bat</code> for the AEHD driver from an elevated prompt, or connect a physical Android phone
PDF report import (RQ1 stretch goal)	Deferred	Text-extraction dependency is in place; wire <code>file_picker</code> → extractor → new record
Staff schedule-template editor	Deferred	Needs a <code>ScheduleTemplate</code> domain repository; templates are currently seeded
Usability study results	Protocol ready	Run the protocol in <code>docs/usability_study.md</code> with 5–8 participants; fill the results table
Live Gemini output	Dormant	Paste a free Google AI Studio key in Admin → AI settings and turn Mock mode off (see <code>docs/ai_setup.md</code>)
Multi-device synchronisation	Out of scope	Future work — the repository layer makes a networked backend a one-class swap

8. Appendix — demo accounts

Seeded on first launch. Password for **every** account: `password` . Full list in `docs/test_accounts.md` .

Role	Login	Count
Admin	<code>admin@myhealth.demo</code>	1
Staff (doctors)	<code>staff1@myhealth.demo</code> ... <code>staff12@myhealth.demo</code>	12
Patients	<code>patient1@myhealth.demo</code> ... <code>patient60@myhealth.demo</code>	60

`patient3@myhealth.demo` is the richest demo patient — most history, abnormal labs, chronic conditions.

Running the app

`flutter run -d windows` · `flutter run -d chrome` · `flutter test` (74 tests) · live AI: `flutter run --dart-define=GEMINI_API_KEY=AIza...`